crawlers, even if they respect this policy, will cause unpredictable behavior.

What can happen is that, if you write your own crawler and you are not careful, it might end up in an endless loop between two or more pages with mutual links (or chain of links) to each other.

To avoid such loops, crawlers need to keep track of pages they already visited. Bloom filters, again, are the best way to do so, as they can store URLs in a compact way, and perform checking and saving of the URLs in constant time.

The price you pay here for false positives is a bit higher than for the previous examples, as the immediate result would be that the crawler will never visit a URL that caused a false positive.

To overcome this issue, it is possible to keep a complete list of the URLs visited in a proper dictionary (or another kind of collection) stored on disk and, if and only if the Bloom filter returns `true`, double check the answer in the dictionary. This approach doesn't allow any space saving, but it provides some savings on the execution time, if there is a high percentage of one-hit-wonders among the URLs.

4.7.4 IO Fetcher

Another area where Bloom-filter-based caching helps the most is reducing the unnecessary fetching/storage of expensive IO resources. The mechanism is the same as with